BST vs AVL Tree

A Comparison Report — Data Structures Assignment

Cairo University | Faculty of Computers and Artificial Intelligence

Marwan | 2025

1. Differences Observed

1.1 Random IDs — Case 1

When I inserted 30 books with random IDs, both trees performed pretty similarly. The BST ended up with a height around 7–8, and the AVL was basically the same or just 1 level less. The search steps were also close — maybe the BST needed 1 or 2 extra steps on some searches, but nothing dramatic.

This makes sense because when IDs are random, the BST naturally stays somewhat balanced. You're not always going left or always going right, so the tree doesn't degrade into a line.

1.2 Sorted IDs — Case 2

This is where the real difference showed up. When I inserted 30 books with IDs 10, 20, 30, ... 300 (all in order), the BST became basically a linked list. Every new node just got added to the right of the previous one, so the height went all the way to 30 — which means searching for the last book took 30 steps, same as just looping through an array.

The AVL Tree on the other hand stayed at a height of around 5. Every time the tree started leaning too much to one side (balance factor became +2 or -2), it did a rotation to fix itself. So searching in the AVL only took around 5 steps max — way better.

Summary Table:

Metric	BST — Random	AVL — Random	BST — Sorted	AVL — Sorted
Height	~7	~6	30	~5
Search Steps (avg)	~6	~5	~15	~4
Search Steps (worst)	~8	~6	30	~5
Balancing	None	Automatic	None	Automatic

2. Why Does This Happen?

A BST has no rules about staying balanced — it just inserts wherever the ID comparison tells it to go. So if you always insert something bigger than the last node, every single node goes to the right, and you get a straight line (degenerate tree). The height becomes O(n) and search becomes O(n) too — which kills the whole point of using a tree.

An AVL Tree fixes this by tracking the balance factor of every node (height of left subtree minus height of right subtree). If this factor ever goes above +1 or below -1, it immediately does a rotation to fix the imbalance. There are 4 rotation cases — LL, RR, LR, RL — and after any of them the tree goes back to being balanced. This guarantees the height stays at O(log n) no matter what order you insert in.

3. When to Use BST vs AVL

Use a BST when:

- You know the data is going to be inserted in a random/unsorted order
- You want simpler code — BST is much easier to implement and debug
- The dataset is small, so the height difference doesn't really matter
- You're doing more insertions/deletions than searches, and you don't want the overhead of rotations every time
- You just need a quick prototype or learning implementation

Use an AVL Tree when:

- The data might come in sorted or near-sorted order (this is the killer case for BST)
- You need guaranteed O(log n) search performance — like in a real library system or database index
- The dataset is large, so a height of 30 vs 5 is a real performance difference
- Search happens way more often than insert/delete, so the rotation cost is worth it
- You need consistent, predictable performance and can't afford worst cases